

ShopEase E-Commerce Platform — Viva Documentation

Table of Contents

1. [Architecture Overview](#)
2. [Frontend \(React + Vite\)](#)
3. [Backend \(PHP + MySQL\)](#)
4. [Database Schema](#)
5. [Context Providers \(Auth, Cart, Wishlist\)](#)
6. [Authentication Flow](#)
7. [Cart Flow \(API + localStorage\)](#)
8. [Wishlist Flow](#)
9. [Products & Product Details](#)
10. [Database Migration](#)

1. Architecture Overview

ShopEase is a **full-stack e-commerce application** with three tiers:

- **Frontend:** React 19 + Vite + React Router v7 (port 5173)
- **Backend API:** PHP 8+ built-in server (port 8000)
- **Database:** MySQL 8.4 (Windows service, auto-start)

The `npm run dev` command runs all three services concurrently using the `concurrently` npm package:

```
concurrently --names "VITE,PHP,MYSQL" "vite" "php -S localhost:8000 -t ../backend ../backend/router.php" "..."
```

Each service gets a named prefix (`[VITE]` , `[PHP]` , `[MYSQL]`) for clear terminal output.

2. Frontend — Key Files

2.1 App.jsx — Root Component

The root component wraps the entire application in providers and sets up routing:

```
<BrowserRouter>
  <AuthProvider>
    <CartProvider>
      <WishlistProvider>
        <Navbar />
        <Routes>...15 routes...</Routes>
        <Footer />
      </WishlistProvider>
    </CartProvider>
  </AuthProvider>
</BrowserRouter>
```

Key point: Provider nesting order matters — `CartProvider` and `WishlistProvider` call `useAuth()` internally, so they must be children of `AuthProvider`.

2.2 Navbar.jsx — Navigation Bar

Imports all three context hooks (`useAuth` , `useCart` , `useWishlist`) to show dynamic badge counts:

- **Cart badge:** `totalItems > 0 && <span className="badge">{totalItems}</span>`
- **Wishlist badge:** `wishlist.length > 0 && <span className="badge">{wishlist.length}</span>`
- **Auth state:** Shows Profile + Logout when logged in; Login link when not
- **Search:** Navigates to `/products?q=searchTerm`

Using Context instead of localStorage hooks eliminates the stale badge problem — all components share one state instance.

2.3 Login.jsx — Login Page

Uses `useAuth().login` which makes an async fetch to the PHP backend:

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError("");
  if (!email || !password) return setError("All fields are required.");
  setLoading(true);
  const result = await login(email, password);
  setLoading(false);
  result.ok ? navigate("/") : setError(result.message);
};
```

Key states: `loading` (shows "Logging in..." text + disables button), `error` (shows auth-error paragraph), and redirect on success.

The button's `disabled={loading}` prevents double-submission.

2.4 `Register.jsx` — Registration Page

Same pattern as Login but collects name, email, phone, password, confirm password:

- Validates all fields required, password ≥ 6 chars, passwords match
- Calls `useAuth().register(name, email, phone, password)`
- On success: navigates to `/login` so user can log in immediately

2.5 `Products.jsx` — Product Listing with Filters

Features:

- **Search:** Filters by name/brand (case-insensitive)
- **Sort:** Price low-to-high, high-to-low, or by rating
- **Brand filter:** Dynamically populated from `[...new Set(products.map(p => p.brand))]`
- **Price range:** 0-25k, 25k-50k, 50k-1L, above 1L
- **Rating filter:** Any, 4+, 4.5+

Uses `useSearchParams` to read search query from URL (set by Navbar search).

2.6 `ProductDetails.jsx` — Product Details Page (NEW)

Route: `/product/:id`

Finds product by ID from static data, then renders:

Back link to Products page

Product name (h1)

Specs table: Uses `Object.entries(product.specs)` to render key-value pairs:

```
<table className="specs-table">
  <tbody>
    {Object.entries(product.specs).map(([key, value]) => (
      <tr key={key}>
        <td className="spec-key">{key}</td>
        <td className="spec-value">{value}</td>
      </tr>
    ))}
  </tbody>
</table>
```

Customer reviews: Static sample data rendered as cards with star ratings

Handles not-found case: shows "Product not found" + back link.

2.7 `ProductCard.jsx` — Product Card Component

Renders a single product with:

- **Image:** Clickable to open `ImageViewer`; fallback on error
- **Discount badge:** Shows e.g. "-10%" if discount > 0
- **Wishlist button:** Toggle heart icon (active/inactive)
- **Rating:** Star characters + numeric value
- **Price:** Discounted price (strikethrough original)
- **View Details:** Link to `/product/:id`
- **Add to Cart:** Calls `onAddToCart` prop from context

2.8 `Cart.jsx` — Cart Page

Features:

- **Empty state:** Shows "Your cart is empty" message
- **Item list:** Image (clickable → `ImageViewer`), name, brand, discount price, quantity controls (+/-), line total, remove button
- **Summary:** Total price + "Proceed to Checkout" button
- Uses `useCart()` for all operations

2.9 `Wishlist.jsx` — Wishlist Page

Same pattern as Cart: empty state, grid of items with image viewer, discount price, remove button.

Uses `useWishlist()`.

2.10 `ImageViewer.jsx` — Lightbox Component

Full-screen image overlay with:

- Click overlay or ✕ button to close
- **Escape key** listener via `useEffect` with `keydown` event

```
useEffect(() => {
  const handleKey = (e) => { if (e.key === "Escape") onClose(); };
  document.addEventListener("keydown", handleKey);
  return () => document.removeEventListener("keydown", handleKey);
}, [onClose]);
```

2.11 `Hero.jsx` — Banner/Slider

Auto-rotating banner with 3 slides, 3-second interval:

```
const [current, setCurrent] = useState(0);
useEffect(() => {
  const timer = setInterval(() => setCurrent((p) => (p + 1) % banners.length), 3000);
  return () => clearInterval(timer);
}, []);
```

- Only `current` slide has `active` class (CSS opacity/transition)
- Dot indicators at bottom — clickable to jump to slide
- Each slide has "Shop Now" and "Offers" buttons

3. Context Providers — State Management

3.1 AuthContext.jsx (**MODIFIED**)

Manages authentication state using React Context + localStorage persistence:

```
const [user, setUser] = useState(() =>
  JSON.parse(localStorage.getItem("currentUser") || "null")
);
```

The **lazy initializer** reads from localStorage so user survives page refresh without an API call.

register() — sends POST to `http://localhost:8000/api/register.php` :

```
const res = await fetch(`${API}/api/register.php`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name, email, phone, password }),
});
const data = await res.json();
if (data.success) return { ok: true, message: data.message };
return { ok: false, message: data.error || "Registration failed" };
```

Returns `{ok, message}` — doesn't auto-login after registration (user goes to /login).

login() — sends POST to `http://localhost:8000/api/login.php` :

```
if (data.success) {
  setUser(data.user); // saves {id, name, email, phone}
  return { ok: true };
}
```

On success, sets `user` state which triggers `useEffect` to persist to localStorage. The `logout` function simply calls `setUser(null)` .

3.2 CartContext.jsx (MODIFIED)

Dual-mode cart: uses **backend API when logged in, localStorage when not**.

Key design decision: Since static `products.js` (25 items, IDs 1-25) and DB products (19 items, IDs 1-19) have completely different data, we can't use `product_id` as a foreign key. Instead, we **store the full product object as JSON** in the `product_data` column.

On mount / user change:

```
useEffect(() => {
  if (user) {
    const local = JSON.parse(localStorage.getItem("cart") || "[]");
    syncCartToAPI(user.id, local).then(() => {
      localStorage.removeItem("cart");
      fetchCart(user.id, setCart);
    });
  } else {
    setCart(JSON.parse(localStorage.getItem("cart") || "[]"));
  }
}, [user?.id]);
```

When user logs in (`user?.id` changes from null to a value), it uploads any local items to the API, clears localStorage, then fetches the server cart.

addToCart():

```
const addToCart = useCallback((product) => {
  if (user) {
    apiCall("POST", { product_id: product.id, quantity: 1, product_data: product });
  } else {
    setCart((prev) => {
      const existing = prev.find((item) => item.id === product.id);
      if (existing) return prev.map((item) =>
        item.id === product.id ? { ...item, qty: item.qty + 1 } : item
      );
      return [...prev, { ...product, qty: 1 }];
    });
  }
}, [user, apiCall]);
```

apiCall() — generic helper that sends HTTP requests and refetches:

```
const apiCall = useCallback(async (method, extra) => {
  if (!user) return;
  await fetch(`${API}/api/cart.php`, {
    method, headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ user_id: user.id, ...extra }),
  });
  fetchCart(user.id, setCart); // refetch to sync
}, [user]);
```

Derived values (computed on every render, not stored in state):

```
const totalItems = cart.reduce((sum, item) => sum + (item.qty || 1), 0);  
const totalPrice = cart.reduce((sum, item) =>  
  sum + (item.qty || 1) * (item.price - (item.price * item.discount) / 100), 0  
);
```

3.3 WishlistContext.jsx (MODIFIED)

Same dual-mode pattern as CartContext. Key differences:

- **toggleWishlist()** — adds if not present, removes if present (toggle behavior)
- POST to `/api/wishlist.php` returns `{inWishlist: true/false}`
- No quantity needed (wishlist items don't have quantity)

```
const toggleWishlist = useCallback((product) => {
  if (user) {
    fetch(`${API}/api/wishlist.php`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ user_id: user.id, product_id: product.id, product_data: product }),
    })
      .then(r => r.json())
      .then(d => { if (d.success) fetchWishlist(user.id, setWishlist); });
  } else {
    setWishlist(prev => {
      const exists = prev.find(item => item.id === product.id);
      if (exists) return prev.filter(item => item.id !== product.id);
      return [...prev, product];
    });
  }
}, [user]);
```

4. Backend — PHP API

4.1 router.php — URL Router

The PHP built-in server (`php -S localhost:8000 -t ../backend ../backend/router.php`) uses this router to handle all requests:

```
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

if ($uri === '/' || $uri === '') {
    require __DIR__ . '/index.php'; // API root
    return true;
}

$file = __DIR__ . $uri;
if (file_exists($file) && pathinfo($file, PATHINFO_EXTENSION) === 'php') {
    require $file; // serve .php files directly
    return true;
}

http_response_code(404);
echo json_encode(['error' => 'Not found']);
return true;
```

This allows clean URLs like `/api/login.php` instead of `/api/login.php?foo=bar` .

4.2 helpers.php — Shared Utilities

Every API file includes this:

- **CORS headers:** Allow requests from `http://localhost:5173` (Vite dev server)
- **OPTIONS preflight:** Return 200 for CORS preflight requests
- **getJsonInput():** Reads JSON from request body: `json_decode(file_get_contents('php://input'), true) ?? []`
- **jsonResponse(\$data):** Echo JSON + exit
- **errorResponse(\$msg, \$code):** Set HTTP status + JSON error + exit
- Includes `db.php` for database connection

4.3 db.php — Database Config

```
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', 'N1nja_Hunt3r');
define('DB_NAME', 'shopease');

function getConnection() {
    $conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($conn->connect_error) {
```

```
http_response_code(500);  
echo json_encode(['error' => 'Database connection failed: ' . $conn->connect_error]);  
exit;  
}  
$conn->set_charset('utf8mb4');  
return $conn;  
}
```

MySQL credentials: root / N1nja_Hunt3r , database shopease .

4.4 login.php — Login Endpoint (NEW)

```
$data = getJsonInput();
$email = trim($data['email'] ?? '');
$password = $data['password'] ?? '';

$stmt = $conn->prepare("SELECT id, name, email, password, phone FROM users WHERE email = ?");
$stmt->bind_param('s', $email);
$stmt->execute();
$result = $stmt->get_result();

$user = $result->fetch_assoc();
if (!password_verify($password, $user['password'])) ErrorResponse('Invalid email or password');

jsonResponse([
    'success' => true,
    'user' => ['id' => $user['id'], 'name' => $user['name'], 'email' => $user['email'], 'phone' => $user['phone']]
]);
```

Uses **prepared statements** (prevents SQL injection) and **password_verify()** (bcrypt). Returns user object without the password hash.

4.5 register.php — Registration Endpoint (NEW)

```
// Check if email already exists
$check = $conn->prepare("SELECT id FROM users WHERE email = ?");
$check->bind_param('s', $email);
$check->execute();
$check->store_result();
if ($check->num_rows > 0) ErrorResponse('Email already registered');

// Hash password and insert
$hashed = password_hash($password, PASSWORD_DEFAULT);
$stmt = $conn->prepare("INSERT INTO users (name, email, password, phone) VALUES (?, ?, ?, ?)");
$stmt->bind_param('ssss', $name, $email, $hashed, $phone);
```

Security features: bcrypt hashing (`password_hash` with `PASSWORD_DEFAULT`), prepared statements, email uniqueness check, email format validation, minimum password length.

4.6 `cart.php` — Cart API (**REWRITTEN**)

Supports 4 HTTP methods with `product_data` JSON storage:

Method	Action	SQL
GET	Fetch user's cart	SELECT product_id, quantity, product_data FROM cart WHERE user_id = ?
POST	Add item (or increase qty)	INSERT ... ON DUPLICATE KEY UPDATE quantity = quantity + ?
PUT	Update quantity	UPDATE cart SET quantity = ?, product_data = ? WHERE user_id = ? AND product_id = ?
DELETE	Remove item	DELETE FROM cart WHERE user_id = ? AND product_id = ?

GET response merges `product_data` with `id` and `qty` :

```
foreach ($rows as $row) {
    $pd = json_decode($row['product_data'], true) ?: [];
    $items[] = array_merge($pd, [
        'id' => (int)$row['product_id'],
        'qty' => (int)$row['quantity'],
    ]);
}
```

The frontend receives items with all original product fields (`name` , `price` , `discount` , `image` , etc.) plus `id` and `qty` .

4.7 `wishlist.php` — Wishlist API (**NEW**)

Same pattern as cart but:

- **POST** is a toggle: checks if item exists → removes if yes, adds if no
- Response includes `inWishlist: true/false`
- No quantity column

```
// Check if already exists
$check = $conn->prepare("SELECT id FROM wishlist WHERE user_id = ? AND product_id = ?");
$check->bind_param('ii', $userId, $productId);
$check->execute();
$check->store_result();

if ($check->num_rows > 0) {
    // Remove (toggle off)
    $del = $conn->prepare("DELETE FROM wishlist WHERE user_id = ? AND product_id = ?");
    $del->bind_param('ii', $userId, $productId);
    $del->execute();
}
```

```

    jsonResponse(['success' => true, 'message' => 'Removed from wishlist!', 'inWishlist' => false]);
  }

  // Add
  $stmt = $conn->prepare("INSERT INTO wishlist (user_id, product_id, product_data) VALUES (?, ?, ?)");
  $stmt->bind_param('iis', $userId, $productId, json_encode($productData));
  $stmt->execute() ? jsonResponse(['success' => true, 'message' => 'Added to wishlist!', 'inWishlist' => true]);

```

4.8 products.php — Products API

Dynamic query builder with filters:

```

if (!empty($_GET['category'])) { $where[] = 'category = ?'; $params[] = $_GET['category']; $types .= 's';
if (!empty($_GET['brand'])) { $where[] = 'brand = ?'; $params[] = $_GET['brand']; $types .= 's';
if (!empty($_GET['max_price'])) { $where[] = 'price <= ?'; $params[] = (float)$_GET['max_price']; $types .=
if (!empty($_GET['search'])) {
    $term = '%' . $_GET['search'] . '%';
    $where[] = '(name LIKE ? OR brand LIKE ?)';
    $params[] = $term; $params[] = $term; $types .= 'ss';
}

```

Dynamic SQL via `match()` for sort, optional LIMIT. Currently not used by frontend (still uses static `products.js`).

4.9 `migrate.php` — Migration Endpoint (NEW)

One-time script that prepares the database for the new cart/wishlist API:

Step 1: Drop FK constraint on `cart.product_id`

Queries `INFORMATION_SCHEMA` to find the auto-generated FK name, then drops it.

Step 2: Add `product_data` JSON column to `cart` table (if not exists).

Step 3: Drop old `wishlist` table and recreate with `product_data` column and no FK on products.

```
POST http://localhost:8000/api/migrate.php
Response: {"success":true,"steps":["Dropped FK constraint...","Added product_data column...","Created wishlist tab
```

5. Database Schema

5.1 Tables

Table	Purpose	Key Columns
users	User accounts	id, name, email (UNIQUE), password (bcrypt hash), phone, address
products	Product catalog (DB version)	id, name, brand, category, price, discount, rating, specifications (JSON), image, stock
cart	Shopping cart	id, user_id (FK→users), product_id, quantity, product_data (JSON)
wishlist	Wishlist	id, user_id (FK→users), product_id, product_data (JSON)
orders	Orders	id, user_id, total_amount, gst, status, address
order_items	Order line items	id, order_id (FK→orders), product_id, quantity, price
messages	Contact form	id, name, email, subject, message
newsletter	Newsletter subscribers	id, email (UNIQUE)

5.2 Cart table — Key Change

Before migration:

```
product_id INT NOT NULL,  
FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
```

After migration:

```
product_id INT NOT NULL,           -- FK constraint removed  
product_data JSON AFTER quantity  -- stores full product object
```

This was necessary because the **static products.js** (used by the frontend) and the **DB products table** have completely different data with different IDs.

6. Authentication Flow

The authentication system uses a **chained flow** between frontend and backend:

```
User enters credentials → Login.jsx → AuthContext.login() → fetch POST /api/login.php
                                ↓
                                PHP: query user by email
                                PHP: password_verify()
                                PHP: return {success, user}
                                ↓
                                AuthContext: setUser(user)
                                → localStorage.setItem("currentUser", ...)
                                → CartContext syncs local→API
                                → WishlistContext syncs local→API
                                ↓
                                Login.jsx: navigate("/")
```

Security features:

- Passwords hashed with `password_hash(PASSWORD_DEFAULT)` = bcrypt
- All SQL uses prepared statements (no string interpolation)
- No password hash returned to frontend — only id, name, email, phone
- CORS restricted to `localhost:5173`

Session persistence: The user object is stored in localStorage under key `currentUser`. On app mount, `AuthContext` reads it with a lazy initializer:

```
const [user, setUser] = useState(() =>
  JSON.parse(localStorage.getItem("currentUser") || "null")
);
```

This means a page refresh doesn't log the user out — they stay logged in until they click Logout (or localStorage is cleared).

7. Cart & Wishlist — Data Flow

Both cart and wishlist use the same dual-mode architecture:

Mode 1: Logged In → API

```
addToCart(product)
  → fetch POST /api/cart.php
    body: { user_id, product_id, quantity, product_data: { id, name, price, image, ... } }
  → PHP inserts into cart table (ON DUPLICATE KEY UPDATE quantity+1)
  → Frontend refetches GET /api/cart.php?user_id=N
  → Cart state updates with merged product + qty fields
```

Mode 2: Not Logged In → localStorage

```
addToCart(product)
  → React state update: [...prev, { ...product, qty: 1 }]
  → useEffect saves to localStorage: localStorage.setItem("cart", JSON.stringify(cart))
```

Login Transition (Sync)

```
User logs in → [user?.id] changes from null to value

CartContext.useEffect:
  1. Read localStorage cart items
  2. syncCartToAPI(): POST each item to /api/cart.php
  3. localStorage.removeItem("cart")
  4. fetchCart(): GET /api/cart.php?user_id=N
  5. setCart(API response)
```

This ensures that items added while not logged in are preserved when the user logs in.

8. All Modified & New Files

File	Status	Description
frontend/src/context/AuthContext.jsx	MODIFIED	Async login/register via PHP API, localStorage persistence
frontend/src/context/CartContext.jsx	MODIFIED	Dual-mode: API when logged in, localStorage fallback
frontend/src/context/WishlistContext.jsx	MODIFIED	Dual-mode toggle wishlist with API sync
frontend/src/pages/ProductDetails.jsx	NEW	Product details with specs table + reviews
frontend/src/pages/Login.jsx	MODIFIED	Async form with loading/error states, API integration
frontend/src/pages/Register.jsx	MODIFIED	Registration with validation + async API call
frontend/src/components/Navbar.jsx	MODIFIED	Context-based badge counts for cart/wishlist
frontend/src/pages/Cart.jsx	MODIFIED	Uses CartContext instead of localStorage hook
frontend/src/pages/Wishlist.jsx	MODIFIED	Uses WishlistContext instead of localStorage hook
frontend/src/pages/Products.jsx	MODIFIED	Uses context hooks for cart/wishlist actions
frontend/src/pages/CategoryPage.jsx	MODIFIED	Uses context hooks for cart/wishlist actions
frontend/src/pages/Profile.jsx	MODIFIED	Reads user from AuthContext, logout button
frontend/src/components/ProductCard.jsx	MODIFIED	Added "View Details" link to /product/:id

frontend/src/components/ImageViewer.jsx	MODIFIED	Escape key listener in useEffect
frontend/src/App.jsx	MODIFIED	Provider nesting (Auth→Cart→Wishlist), ProductDetails route
frontend/src/components/Hero.jsx	MODIFIED	Banner slider with 3s auto-rotation
frontend/src/data/products.js	MODIFIED	Added specs field to all 25 products
frontend/src/styles/global.css	MODIFIED	CSS custom properties for dark theme
frontend/src/hooks/useAuth.js	DELETED	Replaced by AuthContext
frontend/src/hooks/useCart.js	DELETED	Replaced by CartContext
frontend/src/hooks/useWishlist.js	DELETED	Replaced by WishlistContext
backend/api/login.php	NEW	Login with bcrypt password_verify
backend/api/register.php	NEW	Registration with bcrypt password_hash
backend/api/cart.php	REWRITTEN	Full CRUD with product_data JSON storage
backend/api/wishlist.php	NEW	Toggle wishlist with CRUD
backend/api/migrate.php	NEW	DB migration script (FK drop, column add, table create)
backend/api/products.php	NEW	Dynamic product queries with filters
backend/includes/helpers.php	NEW	CORS headers, JSON I/O helpers, DB include
backend/config/db.php	NEW	MySQL connection config (root/N1nja_Hunt3r)

backend/router.php	NEW	PHP built-in server URL router
SQL/ecommerce.sql	MODIFIED	Full schema with 8 tables, 19 sample products
SQL/migrate.sql	NEW	Reference SQL for cart/wishlist migration

9. Common Viva Questions & Answers

Q: Why did you switch from hooks (useAuth.js) to Context API?

A: The old hooks created independent instances of state in each component that imported them. The Navbar badge count was stale because it didn't share the same state as the Cart/Wishlist pages. Context provides a single shared state instance — every consumer gets the same data. This also simplifies the architecture: one Provider at the root, and any component can call `useCart()` etc.

Q: How does the cart sync work when a user logs in?

A: The `useEffect` in `CartContext` watches `user?.id`. When it changes from null to a value (login), it reads `localStorage`, uploads each item to the PHP API via `POST /api/cart.php`, clears `localStorage`, then fetches the server cart. This merges any items added while not logged in with the server state.

Q: Why store product_data as JSON instead of using a foreign key?

A: The frontend uses static `products.js` (25 items with IDs 1-25) while the database has separate products (19 items with different IDs starting from 1). They don't match, so a foreign key on `product_id` would fail. Storing the full product snapshot as JSON in `product_data` means the cart/wishlist are self-contained and don't depend on product IDs matching between frontend and backend.

Q: Explain the security measures in authentication.

A: Three layers: (1) Passwords are hashed with `password_hash(PASSWORD_DEFAULT)` which uses `bcrypt` — never stored in plain text. (2) All SQL uses prepared statements with `bind_param()` — this prevents SQL injection because parameters are sent separately from the query. (3) The API never returns the password hash to the frontend — only id, name, email, phone.

Q: How does the banner/slider animation work?

A: The Hero component uses `useState` for the current slide index (0, 1, or 2) and `useEffect` with `setInterval` at 3000ms (3 seconds). The interval increments the index modulo the number of banners. Each slide div has a conditional `active` class — CSS handles the opacity transition. Dot indicators at the bottom are clickable and call `setCurrent(i)` directly.

Q: What does the PHP router do?

A: The PHP built-in server's router file intercepts all requests. If the URI corresponds to an existing `.php` file (like `/api/login.php`), it requires that file. If it's the root (`/`), it shows the API

status. Otherwise, it returns a 404 JSON response. This enables clean API URLs without a traditional web server like Apache.

Q: How does the image viewer component work?

A: ImageViewer renders a full-screen overlay with the clicked image. A `useEffect` registers a `keydown` event listener that calls `onClose()` when Escape is pressed. Clicking the overlay background or the ✕ button also closes it. Clicking the image itself calls `e.stopPropagation()` to prevent the overlay click from closing it.

— End of Documentation —